Finally, you amend the request handler to take the service as a parameter and the route handler to utilize it to perform the calculation.

Listing 11.23 The request handler taking the `BookingService` as a parameter

```
 app.MapPost("/api/property/booking",
    (BookingInputModel model, IBookingService bookingService) =>   ❶
{
    var booking = new Booking {                                    ❷
        StartDate = model.StartDate.Value,                         ❷
        EndDate = model.EndDate.Value,                             ❷
        NumberOfGuests = model.NumberOfGuests,                     ❷
        DayRate = model.Property.DayRate                           ❷
    };                                                             ❷
    var totalCost = bookingService.Calculate(booking);             ❸
    return Results.Ok(new { TotalCost = totalCost });
});
```

❶ The handler takes the IBookingService as a parameter.

❷ An instance of the Booking class is instantiated from the input model.

❸ It is passed to the service as a parameter, and the returned value is sent back to the client.

In this example, the binding sources are inferred. The framework will search for them from all sources, until a matching one is found. To make them explicit, and thereby narrow the search, you add `From*` attributes accordingly:

```
app.MapPost("/api/property/booking",
    ([FromBody]BookingInputModel model,
     [FromServices]IBookingService bookingService)
```

11.3.3 Minimal API return types

In the example so far, you have used the `Results.Ok` method to return the data serialized to JSON with a 200 status code. This is one of several